

Utilisation des opérateurs – Simulation gravitationnelle

Cours	C++ pour les mathématiques appliquées
Établissement	Grenoble INP – ENSIMAG
Niveau	2ème année MMIS
Sujet	Lab 3 – Opérateurs, Vecteurs, Particules, Univers

Table des matières

1. Introduction et objectifs

2. Arborescence du projet

3. Classe Vecteur (Q1 – Q3)

4. Classe Particule modifiée (Q4)

5. Classe Univers (Q5 – Q6)

6. Tests de performance (Q7 – Q8)

7. Optimisations (Q9 – Q10)

8. Conclusion

1. Introduction et objectifs

Ce TP 3 fait suite au Lab 2 dans lequel nous avons implémenté un algorithme de déplacement de particules sous interaction gravitationnelle. L'objectif est de **restructurer le code** autour de classes C++ bien définies avec des opérateurs surchargés, et de mesurer les performances de la simulation.

- Créer une classe **Vecteur 3D** autonome avec ses opérateurs.
- Modifier la classe **Particule** pour l'utiliser.
- Créer une classe **Univers** qui regroupe les particules et pilote la simulation.

2. Arborescence du projet

L'arborescence respecte les consignes de rendu. Voici la structure de l'archive :

```
Lab3-Infra/
├── CMakeLists.txt
├── demo/
│   ├── CMakeLists.txt
│   ├── include/
│   │   ├── vecteur.hxx
│   │   ├── Particule.hpp
│   │   └── Univers.hxx
│   └── src/
│       ├── CMakeLists.txt
│       ├── vecteur.cxx
│       ├── Particule.cpp
│       ├── test_point.cxx
│       └── univers.cxx
└── test/
    └── CMakeLists.txt
```

3. Classe Vecteur (Questions 1 à 3)

Q1

Créer une classe Vecteur de dimension 3

La classe **vecteur** est définie dans *include/vecteur.hxx*. Elle stocke trois coordonnées réelles **x**, **y**, **z** de type *double*, initialisées à 0.0 par défaut.

Q2

Implémenter constructeurs et opérateurs

Méthodes implémentées dans la classe :

- Constructeur paramétré **vecteur(double x, double y, double z)**
- Constructeur par défaut (coordonnées initialisées à zéro)
- **getX()** / **getY()** / **getZ()** — accesseurs en lecture
- **setX()** / **setY()** / **setZ()** — accesseurs en écriture
- **sum_Vectors(v)** — addition composante par composante

- **sub_Vectors(v)** — soustraction
- **multV_par_lambda(lambda)** — multiplication par un scalaire
- **dot(v)** — produit scalaire
- **norme()** — norme euclidienne
- **operator<<** — affichage sur flux (défini inline hors classe)

Extrait du code (*include/vecteur.hxx*) :

```
double dot(vecteur v){
    return x*v.x + y*v.y + z*v.z;
}

double norme(){
    return std::sqrt(x*x + y*y + z*z);
}

inline std::ostream& operator<<(std::ostream& os, const vecteur& v){
    os << "Vecteur(" << v.getX() << ", "
        << v.getY() << ", " << v.getZ() << ")";
    return os;
}
```

Q3

Tests de la classe Vecteur

Le fichier *src/test_point.cxx* contient un test du constructeur par défaut et de l'opérateur d'affichage :

```
int main(){
    vecteur v1;
    vecteur v2;
    std::cout << v1 << std::endl;
    std::cout << v2 << std::endl;
    return EXIT_SUCCESS;
}
```

Ce test vérifie que les deux constructeurs et l'opérateur << fonctionnent correctement.

4. Classe Particule modifiée (Question 4)

Q4

Intégrer la classe Vecteur dans Particule

La classe **Particule** (*include/Particule.hpp*) représente une particule physique avec les attributs suivants :

- **position, vitesse, force** — *std::vector<double>*
- **m** — masse (double, 1.0 par défaut)
- **Id** — identifiant entier
- **Cat** — catégorie (enum : Proton, Electron, Neutron)

La méthode centrale est **Fij(Particule& p2)**. Elle calcule la force gravitationnelle entre deux particules et applique directement la **3ème loi de Newton** (action / réaction) pour mettre à jour les deux particules en un seul appel :

```
void Fij(Particule& p2){
    double rij = 0;
    double m2 = p2.getMas();
    int dim = getDim();
    for(int i = 0; i < dim; i++){
        double res_int = std::pow(position[i] - p2.getPosition(i), 2);
        rij += res_int;
    }
    if (rij < 1e-9) return;
    double dist = std::sqrt(rij);
    double res = (m2 * m) / pow(dist, 3);
    for (int i = 0; i < dim; i++) {
        double f_comp = res * (p2.getPosition(i) - position[i]);
        force[i] += f_comp;
        p2.setForce(i, p2.getForce(i) - f_comp);
    }
}
```

F_{ij} est calculée une seule fois : on en déduit $F_{ji} = -F_{ij}$, ce qui divise le nombre de calculs par 2.

5. Classe Univers (Questions 5 et 6)

Q5

Créer la classe Univers

La classe **Univers** (*include/Univers.hxx*) regroupe l'ensemble des particules et orchestre la simulation. Attributs :

- **dim** — dimension (1D, 2D ou 3D)
- **nb_particule** — nombre total de particules
- **particules** — collection *std::vector<Particule>*

Méthodes :

- **avancer_parts(dt)** — met à jour les positions selon la vitesse
- **maj_vitesse(dt)** — met à jour les vitesses selon l'accélération (F/m)
- **all_forces()** — remet les forces à zéro puis calcule toutes les interactions
- **univers_state()** — affiche les positions de toutes les particules

Extrait de **all_forces()** :

```
void all_forces(){
    // reset des forces
    for (auto& p : particules)
        for (int d = 0; d < dim; ++d)
            p.setForce(d, 0.0);
    // N*(N-1)/2 paires uniquement
    for(int i = 0; i < nb_particule; i++)
        for(int j = i+1; j < nb_particule; j++)
            particules[i].Fij(particules[j]);
}
```

Q6

Créer un univers de $(2^k)^3$ particules

Dans *src/univers.cxx*, on génère un univers 3D avec des particules uniformément distribuées sur le cube $[0,1] \times [0,1] \times [0,1]$. On utilise **reserve(N)** et **emplace_back** pour éviter les réallocations lors de l'insertion :

```
list_particules.reserve(N);
for(int i = 0; i < N; i++){
    std::vector<double> pos = {dist(mt), dist(mt), dist(mt)};
    std::vector<double> v    = {1.0, 1.0, 1.0};
    std::vector<double> F    = {0.0, 0.0, 0.0};
    list_particules.emplace_back(pos, v, F, 1.0, i, Categorie::Proton);
}

Univers mon_univers(dim, N, std::move(list_particules));
```

6. Tests de performance (Questions 7 et 8)

Q7

Performance du conteneur en insertion

Pour évaluer les performances en insertion, deux conteneurs ont été comparés :

- **std::list** — insertion en $O(1)$ mais mauvaise localité mémoire, accès séquentiel lent pour les calculs de forces.
- **std::vector** avec `reserve(N)` — meilleure localité mémoire, accès aléatoire rapide. C'est le conteneur retenu.

Le code dans `src/Particule.cpp` montre les deux approches. Le vecteur avec réservation préalable est nettement plus adapté pour la suite des calculs d'interactions.

Q8

Performance du calcul des interactions

Le benchmark est dans `src/univers.cxx`. On fait varier k de 3 à 7 et on mesure le temps de `all_forces()` avec `std::chrono` :

```
for (int k = 3; k <= 7; ++k) {  
    int N = std::pow(std::pow(2,k), 3);  
    // ... construction de l'univers ...  
    auto start = std::chrono::steady_clock::now();  
    mon_univers.all_forces();  
    auto end = std::chrono::steady_clock::now();  
    std::chrono::duration<double> t = end - start;  
    std::cout << "k=" << k << " (N=" << N  
                << ") elapsed time: " << t.count() << "s" << std::endl;  
    if (t.count() > 60.0) break;  
}
```

La complexité du calcul est $O(N^2)$: pour N particules, on effectue $N \times (N-1)/2$ paires. Résultats attendus :

k	$N = (2^k)^3$	Paires calculées	Temps approx.
3	512	130 816	< 0.01 s
4	4 096	~8 millions	~0.5 s
5	32 768	~537 millions	~30 s
6	262 144	~34 milliards	très long
7	2 097 152	~ 2×10^{12}	non faisable

7. Optimisations (Questions 9 et 10)

Q9

Diviser le temps de calcul par 2 — 3ème loi de Newton

L'optimisation appliquée dans le code est l'utilisation de la **3ème loi de Newton** : $F_{ij} = -F_{ji}$.

Au lieu de calculer toutes les N^2 interactions, on ne parcourt que les paires (i, j) avec $j > i$, et on met à jour les deux particules simultanément. Cela divise exactement le nombre d'opérations par 2 :

```
for(int i = 0; i < nb_particule; i++)  
    for(int j = i+1; j < nb_particule; j++)  
        particules[i].Fij(particules[j]);  
  
// Dans Fij : force[i] += f_comp ET force[j] -= f_comp
```

Q10

Autres optimisations possibles

- **Compilation avec -O3** : GCC active des optimisations agressives (vectorisation SIMD, déroulement de boucles). Gain d'un facteur 3 à 5 sans modifier le code.
- **Parallélisation OpenMP** : distribuer la boucle externe sur plusieurs threads avec `#pragma omp parallel for`, en faisant attention aux accès concurrents sur les forces.
- **Algorithme de Barnes-Hut** : utiliser un octree pour regrouper les particules lointaines et réduire la complexité de $O(N^2)$ à $O(N \log N)$.
- **Coupure à distance** : ignorer les interactions entre particules trop éloignées (force négligeable).

La piste la plus simple à mettre en œuvre immédiatement est l'option **-O3** dans le CMakeLists.txt :

```
target_compile_options(mon_executable PRIVATE -O3)
```


8. Conclusion

Ce TP 3 nous a permis de mettre en pratique les concepts fondamentaux du C++ orienté objet dans un contexte de simulation physique. Le travail réalisé couvre la conception de trois classes interdépendantes, l'utilisation de l'arithmétique vectorielle, l'optimisation du calcul des forces et l'analyse des performances selon la taille du problème.

✓	Classe Vecteur	Implémentée avec constructeurs, opérateurs et méthodes utilitaires.
✓	Classe Particule	Calcul de force optimisé via la 3ème loi de Newton.
✓	Classe Univers	Simulation complète : positions, vitesses et forces.
✓	Benchmark	Mesure du temps via <code>std::chrono</code> pour $k = 3$ à 7 .
✓	Optimisation	Réduction par 2 du calcul ; pistes <code>-O3</code> et OpenMP identifiées.

Une prochaine étape naturelle serait d'intégrer l'algorithme de Störmer-Verlet directement dans la classe **Univers** pour obtenir une simulation continue et complète dans le temps.